# 3_Extracción_de_características

February 24, 2026

[26]:
```
import pandas as pd
import numpy as np

df_train = pd.read_csv('../partitions/train.csv', sep=';')
df_test = pd.read_csv('../partitions/test.csv', sep=';')

matriz_datos_train = feature_extraction(df_train)
matriz_datos_test = feature_extraction(df_test)

# print(np.shape(matriz_datos_test))

import os
if not os.path.exists('../features'):
    os.mkdir('../features')

np.save('../features/matriz_datos_train.npy', matriz_datos_train)
np.save('../features/matriz_datos_test.npy', matriz_datos_test)
```

[20]:
```
def feature_extraction(df):

    import cv2
    import matplotlib.pyplot as plt

    fingerprint = []
    for i in range(0, len(df)):
#         print('[INFO] --- Extrayendo información para la muestra ', str(i))
        file = df.ID[i]
        img = cv2.imread('../Material/Images/' + file)
        rnfl_mask = cv2.imread('../Material/RNFL_masks/' + file, 0)
        retina_mask = cv2.imread('../Material/Retina_masks/' + file, 0)

    #     # Visualización
    #     fig, ax = plt.subplots(1,3)
    #     ax[0].imshow(img, cmap='gray'), ax[0].set_title('Imagen')
    #     ax[1].imshow(rnfl_mask, cmap='gray'), ax[1].set_title('RNFL')
    #     ax[2].imshow(retina_mask, cmap='gray'), ax[2].set_title('Retina')

        # ESTADÍSTICOS UNIDIMENSIONALES en la RNFL
```

```
        thickness_rnfl = []
        for j in range(0, rnfl_mask.shape[1]):
            pos = np.where(rnfl_mask[:,j]==255)
            thickness_rnfl.append(pos[0][-1]-pos[0][0])
        thickness_rnfl = np.array(thickness_rnfl)
    #     print(np.shape(thickness_rnfl))

        # Características basadas en medidas de tendencia central
        media = np.mean(thickness_rnfl)
        mediana = np.median(thickness_rnfl)

        # Características basadas en medidas de dispersión
        desvest = np.std(thickness_rnfl)

        # Características de distribución
        from scipy import stats
        asimetria = stats.skew(thickness_rnfl)
        curtosis = stats.kurtosis(thickness_rnfl)

        # Otras características
        minimo = np.min(thickness_rnfl)
        maximo = np.max(thickness_rnfl)

        # (fingerprint RNFL)
        features_RNFL = [media, mediana, desvest, asimetria, curtosis, minimo,␣
↪maximo] # estadísticos unidimensionales

        # CARACTERÍSTICAS BIDIMENSIONALES en la estructura de la RETINA
        from skimage.measure import regionprops
        prop = regionprops(retina_mask)
        bb = prop[0].bbox
        retina = img[bb[0]:bb[2], bb[1]:bb[3], 0]

    #     plt.imshow(retina, cmap='gray')
    #     plt.show()

        # Gray-Level Coocurrence Matrix (GLCM)
        from skimage.feature import greycomatrix, greycoprops
        GLCM = greycomatrix(retina, distances=[2], angles=[90], levels=256,␣
↪symmetric=True, normed=True)
        contraste = greycoprops(GLCM, 'contrast')[0,0]
        disimilitud = greycoprops(GLCM, 'dissimilarity')[0,0]
        homogeneidad = greycoprops(GLCM, 'homogeneity')[0,0]
        ASM = greycoprops(GLCM, 'ASM')[0,0]
        energia = greycoprops(GLCM, 'energy')[0,0]
        correlacion = greycoprops(GLCM, 'correlation')[0,0]
```

```
        # Local Binary Patterns (LBP)
        from skimage.feature import local_binary_pattern
        R=1 # radio
        P=8*R # vecinos
        lbp_image = local_binary_pattern(retina, P, R, method='uniform')

        lbp_image = np.uint8(lbp_image)

        hist_lbp = cv2.calcHist([lbp_image.ravel()], [0], None, [P+2], [0, P+2])
        hist_lbp = hist_lbp.astype('float')
        hist_lbp /= (hist_lbp.sum() + 1e-7)
        hist_lbp = hist_lbp.tolist()
        hist_lbp = [item for sublist in hist_lbp for item in sublist]

        # Visualización de la imagen lbp y el histograma
    #     plt.imshow(retina, cmap='gray')
    #     plt.show()

    #     plt.imshow(lbp_image, cmap='gray')
    #     plt.show()

    #     plt.plot(hist_lbp)
    #     plt.grid(True)
    #     plt.show()

        # Características de textura (fingerprint retina)
        features_Retina = [contraste, disimilitud, homogeneidad, ASM, energia,␣
 ↪correlacion] + hist_lbp # Características bidimensionales

        # Extraer la información de la clase
        if df.Class[i] == 'Healthy':
            etiqueta = [0]
        else:
            etiqueta = [1]

        fingerprint.append(features_RNFL + features_Retina + etiqueta)
    #     print(np.shape(fingerprint))

    matriz_datos = np.array(fingerprint)
#     print(np.shape(matriz_datos))

    return matriz_datos
```